with radius equal to the distance to the current guess for the nearest neighbor) – or, said otherwise, if the a branch is close enough to the target point to be traversed. Also, while in k-d trees we check and update distance at every node, Ss-trees (and R-trees) only host points in their leaves, so we will only update the initial distance after we traverse the tree to the first leaf.

To improve search performance, it's important to traverse branches in the right order. While it is not obvious to decide what this order is, a good starting point is sorting nodes based on their minimum distance from the query point. It is not guaranteed that the node that *potentially* has the closest point (meaning its bounding envelope is closest to the target point) will *actually* have a point so close to our target, and so we can't be sure this heuristic will produce the best ordering possible; however, on average it helps, compared to following a random order.

To remind you why this is important: we have discussed in section 9.3.5 how getting to a better guess of the nearest neighbor distance helps us pruning more branches, and thus improving search performance. If we know, in fact, that there is a point within distance `D` from our query point, we can prune all branches whose bounding envelopes are further than `D`.